

NumPy Array Reshaping

[< Previous](#)[Next >](#)

Reshaping arrays

Reshaping means changing the shape of an array.

The shape of an array is the number of elements in each dimension.

By reshaping we can add or remove dimensions or change number of elements in each dimension.

Reshape From 1-D to 2-D

Example

[Get your own Python Server](#)

Convert the following 1-D array with 12 elements into a 2-D array.

The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8,
               9, 10, 11, 12])

newarr = arr.reshape(4, 3)

print(newarr)
```

[Try it Yourself »](#)

Reshape From 1-D to 3-D

Example

Convert the following 1-D array with 12 elements into a 3-D array.

The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8,
               9, 10, 11, 12])

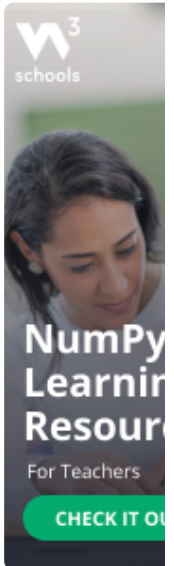
newarr = arr.reshape(2, 3, 2)

print(newarr)
```

[Try it Yourself »](#)

ADVERTISEMENT

[REMOVE ADS](#)



COLOR
PICKER



[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C](#) [»](#)

NumPy Tutorial

[NumPy HOME](#)[NumPy Intro](#)[NumPy Getting Started](#)[NumPy Creating Arrays](#)[NumPy Array Indexing](#)[NumPy Array Slicing](#)[NumPy Data Types](#)[NumPy Copy vs View](#)[NumPy Array Shape](#)[NumPy Array Reshape](#)[NumPy Array Iterating](#)[NumPy Array Join](#)[NumPy Array Split](#)[NumPy Array Search](#)[NumPy Array Sort](#)[NumPy Array Filter](#)

NumPy Random

[Random Intro](#)[Data Distribution](#)[Random Permutation](#)[Seaborn Module](#)[Normal Distribution](#)[Binomial Distribution](#)[Poisson Distribution](#)[Uniform Distribution](#)[Logistic Distribution](#)[Multinomial Distribution](#)[Exponential Distribution](#)[Chi Square Distribution](#)[Rayleigh Distribution](#)[Pareto Distribution](#)

Shape?

Yes, as long as the elements required for reshaping are equal in both shapes.

We can reshape an 8 elements 1D array into 4 elements in 2 rows 2D array but we cannot reshape it into a 3 elements 3 rows 2D array as that would require $3 \times 3 = 9$ elements.

Example

Try converting 1D array with 8 elements to a 2D array with 3 elements in each dimension (will raise an error):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(3, 3)

print(newarr)
```

[Try it Yourself »](#)

Returns Copy or View?

Example

Check if the returned array is a copy or a view:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

print(arr.reshape(2, 4).base)
```

[Try it Yourself »](#)

The example above returns the original array, so it is a view.

Unknown Dimension

You are allowed to have one "unknown" dimension.

Meaning that you do not have to specify an exact number for one of the dimensions in the reshape method.

Pass `-1` as the value, and NumPy will calculate this number for you.

Example

Convert 1D array with 8 elements to 3D array with 2x2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(2, 2, -1)

print(newarr)
```

[Try it Yourself »](#)

Note: We can not pass `-1` to more than one dimension.

Flattening the arrays

Flattening array means converting a multidimensional array into a 1D array.

We can use `reshape(-1)` to do this.

Example

Convert the array into a 1D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)
```

Try it Yourself »

Note: There are a lot of functions for changing the shapes of arrays in numpy `flatten`, `ravel` and also for rearranging the elements `rot90`, `flip`, `fliplr`, `flipud` etc. These fall under Intermediate to Advanced section of numpy.

Exercise [?]

Consider the following code: